TaskFlow — Take-Home Assignment

Full-Stack Developer

📅 Due in 3 days ⚛ React (JS or TS) 🟢 Node.js or Python (your choice)

> **Submission deadline:** within 3 days of receiving this.

1. The scenario

You're joining a small team building **TaskFlow**, a simple task board for small teams — think a lightweight version of Trello. We want to see how you approach building a small full-stack feature from scratch: how you structure your code, handle data, and think about the details that separate "it works on my machine" from "it works."

2. Core requirements (must-have)

2.1 Data model

- A **Board** contains **Columns** (e.g. "To Do", "In Progress", "Done").
- A **Column** contains **Tasks**.
- Each **Task** has: title, description (optional), a status (which column it's in), a priority (Low / Medium / High), and a created date.

2.2 Core features

- View a board with its columns and tasks.
- Create a new task (title required, description and priority optional).
- Edit an existing task (title, description, priority).
- Delete a task.
- Move a task from one column to another — either via drag-and-drop, or via a simple control (like a dropdown or buttons) if drag-and-drop feels like too much for the time budget. Drag-and-drop is nicer to see, but a working dropdown beats a broken drag-and-drop.
- All changes should be saved to a real backend + database (not just local state) — reloading the page should show the same data.

2.3 Basic filtering

- Let the user filter the visible tasks by priority (e.g. "show only High priority").
- A simple text search box that filters tasks by title is a nice-to-have but not required.

2.4 Basic validation & error handling

- Don't allow creating a task with an empty title (enforce this on the backend too, not just the form).
- If a request to the backend fails, show the user something reasonable (not a blank screen or a raw error in the console).

2.5 Database — this part is deliberately checked closely

We want to see that you can design and query a real relational database yourself, not just call an ORM's default methods.

- Use a real database (SQLite is completely fine — you don't need Postgres/MySQL running separately, though either is also fine if you prefer).
- Include your schema as an actual file we can read: a `schema.sql`, your ORM's migration files, or a `CREATE TABLE` block in your README. It should have sensible types, a primary key on each table, a foreign key from `Task` → `Column` (and `Column` → `Board`), and `NOT NULL` on required fields like title.
- Write at least **two queries that aren't simple "get all rows"** — e.g. "count of tasks per column on a board" and "tasks with a given priority, newest first." Show us the actual SQL (or query-builder code), not just the JSON result.
- Include a small seed script or seed data so a fresh database isn't empty on first run.

3. Non-functional requirements

- **Runs locally with clear instructions:** README should explain how to install dependencies and start both the frontend and backend from a fresh clone. `docker-compose up` is great if you're comfortable with it, but plain `npm install && npm run dev`-style instructions are completely fine too.
- **Tests:** a handful of backend tests. At minimum: (1) creating a task with no title fails, (2) moving a task updates its status/column correctly, and (3) one test that hits the database layer directly (e.g. the "tasks per column" or "tasks by priority" query returns the right rows for known seed data).
- **Clean, readable code:** sensible file/folder structure, meaningful names, and no leftover commented-out code or console.logs from debugging. You don't need a fancy architecture — just one that's easy to follow.

4. Explicitly out of scope (please don't spend time here)

To keep this within the time budget, please skip: user accounts/login, multiple users or teams, real-time updates between browser tabs, file uploads, and visual design polish beyond "looks reasonably tidy." A plain but functional UI is completely fine — this is not a design evaluation.

5. Stretch goals (optional — only if the core above is fully working)

Pick at most one, and only if you have time left over:

- Drag-and-drop (if you did the dropdown/button version for the core requirement).
- Text search by task title.
- A simple "task count per column" shown in each column header.

Do not attempt a stretch goal at the expense of a working core — a smaller, solid submission is scored higher than a bigger, broken one.

Note: deploying your project to a live server (Render, Railway, Vercel, Fly.io, or any host of your choice) so we can open a working link directly — candidates who do this will be given priority.

6. What to submit

Please submit within **3 days** of receiving this assignment, through the **Internshala** application/assignment portal for this role.

1. A link to a public (or invite-accessible) Git repository with your code.
2. A short section in your README (a few sentences is fine) covering: any decisions or assumptions you made that weren't spelled out above; what you'd improve or add if you had more time; roughly how long you spent; and — optional, but we like reading these — one thing you looked up, learned, or found genuinely interesting while building this. No wrong answer here; we're just curious what caught your attention.
3. Setup instructions that work from a clean clone — please try running your own instructions once before submitting, on a fresh checkout if possible.

7. How we'll evaluate this

Roughly in this order:

1. **Does it work?** Can we clone it, follow your instructions, and actually create/edit/move/delete tasks with data that persists?
2. **Database & code quality** — is the schema sensible (types, keys, constraints)? Are the two required queries actually querying the database rather than filtering everything after fetching it all? Is the code organized in a way another developer could understand and extend?

3. **Attention to detail** — validation, error handling, and the small things (does an empty title actually get rejected? does the UI recover from a failed request?).
4. **Communication & curiosity** — are your assumptions and trade-offs clearly written down? Does your write-up show you were thinking about *why*, not just typing until it worked?

We are not scoring on feature volume or visual design. A clean, working, well-tested task board beats a feature-packed but fragile one. If we move forward, we'll also ask you to walk us through a few of your own decisions live — so it's worth actually understanding the choices you made, not just getting them to pass.

8. Questions

If anything here is unclear, make a reasonable assumption, write it down in your README, and keep going — figuring out what to do with ambiguity is a normal part of the job, and we'd rather see how you handle it than have every detail spelled out.

Good luck, and thank you for the time you put into this.